



Especificación de Requisitos de Software (SRS)

Indexación Espacial con QuadTree para Monitoreo del Río Zamora

Selena Castillo; Cristhian Davila; Elian Jiménez; Carlos Martínez; Emily Salas

Carrera de Computación

7 de octubre de 2026

Datos generales

Dato	Detalle
Asignatura	Estructura de Datos
Proyecto	ABP (Grupo 1)
Docente	Cristian Ramiro Narváez Guillén
Institución / Carrera	Computación
Ciclo / Paralelo	3 Ciclo
Fecha de entrega	7 de octubre de 2026

1. Introducción

1.1. Propósito

Este documento de Especificación de Requisitos de Software (SRS) analiza el problema planteado en el proyecto ABP y define las necesidades [1] de un sistema orientado a la gestión de datos estáticos y dinámicos en Node.js. Está dirigido al docente, quien evalúa el análisis realizado, y al equipo de desarrollo (Grupo 1), que lo usará como base para el diseño e implementación posteriores.

1.2. Alcance

El sistema, denominado *Indexación Espacial con QuadTree para Monitoreo del Río Zamora*, apoyará el monitoreo ambiental de la cuenca del río Zamora (Loja) [2]. Su núcleo es un índice espacial 2D basado en QuadTree, que organiza las lecturas geolocalizadas de sensores y permite recuperarlas con eficiencia.

El sistema hará:

- Almacenar puntos de monitoreo y lecturas ambientales, georreferenciados por latitud y longitud.
- Indexar los puntos mediante un QuadTree para acelerar las búsquedas espaciales.
- Atender consultas por delimitación geográfica (Range Query): devolver los puntos contenidos en una región rectangular definida por el usuario.
- Gestionar de forma explícita los datos estáticos (ubicación de las estaciones) y los dinámicos (lecturas que se insertan o actualizan en el tiempo) sobre Node.js.

El sistema no hará:

- Reemplazar un sistema de información geográfica (SIG) completo ni generar mapas cartográficos avanzados.
- Adquirir datos directamente del hardware de los sensores; se trabajará con datos simulados o precargados.
- Realizar modelado hidrológico ni predicción ambiental.

1.3. Definiciones, acrónimos y abreviaturas

Término	Definición
QuadTree	Estructura de árbol que subdivide recursivamente el espacio 2D en cuatro cuadrantes (NW, NE, SW, SE).
Range Query	Búsqueda de los puntos contenidos en una región rectangular o poligonal.
Call Stack (Pila)	Zona de memoria que guarda tipos primitivos, punteros y el seguimiento de las llamadas a funciones.
Memory Heap (Montón)	Zona de memoria donde residen objetos, arreglos, funciones e instancias de clases.
V8	Motor de JavaScript de Google, escrito en C++, usado por Chrome y Node.js.
GC	Garbage Collector (recolector de basura): libera automáticamente la memoria del Heap que ya no es alcanzable.
SRS	Software Requirements Specification (Especificación de Requisitos de Software).

Tabla 1: Definiciones, acrónimos y abreviaturas.

1.4. Marco teórico: gestión de memoria en V8

1.4.1. Pila (Call Stack)

El Call Stack es una estructura LIFO (Last-In, First-Out), muy rápida, pequeña y de tamaño limitado [3]. Actúa como la “lista de pendientes” del motor: por cada función invocada se añade (push) un marco, y al terminar se elimina (pop), liberando de inmediato

sus variables locales. Almacena los valores primitivos (number, boolean, string, null, undefined), por tener tamaño fijo y conocido, y los punteros que apuntan a objetos del Heap. Si una recursión es demasiado profunda o no tiene caso base, se produce un Stack Overflow (RangeError: Maximum call stack size exceeded). Este punto es especialmente importante en el QuadTree, cuyas inserciones y búsquedas son recursivas.

1.4.2. Montón (Memory Heap)

El Memory Heap es una región amplia, dinámica y mayormente no estructurada, que actúa como “almacén” de datos complejos cuyo tamaño no se conoce de antemano o puede variar durante la ejecución. Allí residen los objetos, arreglos, funciones e instancias de clases. La pila no contiene estos datos, sino solo una referencia hacia ellos. Un dato permanece en el Heap mientras exista al menos una referencia activa desde la pila o el ámbito global. El Garbage Collector usa el algoritmo Mark and Sweep: marca los objetos alcanzables desde las raíces y elimina los no marcados. Si quedan referencias olvidadas, como variables globales o temporizadores `setInterval`, los datos no se liberan y se produce una fuga de memoria (Memory Leak), un riesgo real en un sistema de monitoreo que recibe lecturas continuamente [4].

1.4.3. Comparación Stack vs Heap

Aspecto	Pila (Call Stack)	Montón (Memory Heap)
Qué almacena	Marcos de funciones, valores primitivos y punteros.	Objetos, arreglos, funciones e instancias de clases.
Velocidad de acceso	Muy rápida; estructura LIFO ordenada.	Más lenta; estructura no ordenada y dinámica.
Gestión	Automática e inmediata al terminar cada función.	Gestionada por el Garbage Collector (Mark and Sweep).
Ejemplo en el proyecto	<code>lat</code> , <code>lng</code> como number; banderas boolean.	Nodo del QuadTree con hijos NW/NE/SW/SE y arreglo de lecturas de sensores.

Tabla 2: Comparación entre Stack y Heap.

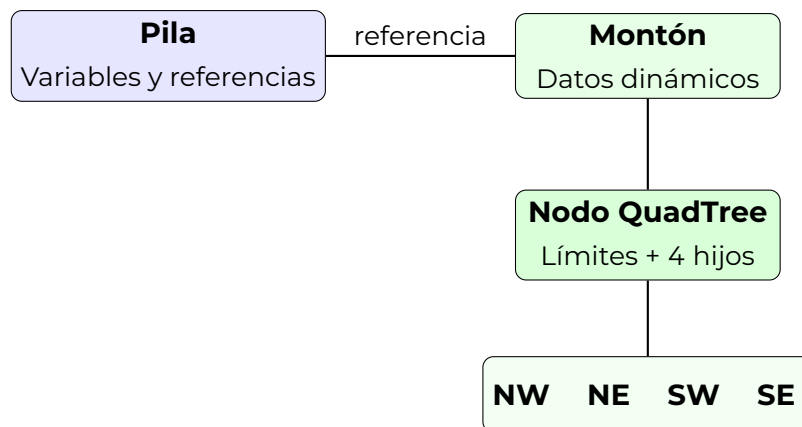


Figura 1: Representación simplificada de la memoria y estructura QuadTree

2. Descripción general

2.1. Perspectiva del producto

Este sistema de Monitoreo para el Río Zamora viene a ser una aplicación web independiente la cual funcionará como una torre de control ambiental, la cual procesa la telemetría de una red de estaciones meteorológicas y pluviométricas distribuidas a lo largo de la cuenca, delimitada hasta la Central Hidroeléctrica Delsitanisagua. También contará con un panel interactivo accesible desde un navegador. Su arquitectura desacopla un backend en Node.js (motor V8), que organiza y procesa al instante grandes volúmenes de telemetría mediante la subdivisión espacial de un QuadTree en memoria, y un frontend en Next.js con Leaflet y D3.js que traduce estos datos en mapas dinámicos y gráficas en tiempo real, sirviendo tanto a analistas técnicos y gestores de riesgo para consultas avanzadas como a la ciudadanía para la supervisión preventiva del caudal.

2.2. Funciones principales del producto

1. **Recibir datos del clima al instante:** recibe y actualiza periódicamente las lecturas de lluvia de las estaciones del río Zamora a partir de registros precargados o secuencias simuladas en tiempo real.
2. **Organizar el mapa con rapidez (QuadTree):** divide la cuenca en cuadros más pequeños en la memoria de la computadora para ordenar los datos sin que el sistema se vuelva lento.
3. **Búsqueda espacial por delimitación geográfica (Range Query):** permite al usuario marcar un área o polígono en el mapa y recuperar al instante solo las estaciones y datos de esa zona, descartando todo lo que queda afuera.
4. **Mostrar un mapa interactivo (Leaflet):** presenta el recorrido del río, sus quebradas y la ubicación de las estaciones en una pantalla web fácil de explorar y acercar con el cursor.

5. **Generar gráficos y alertas sencillas (D3.js):** dibuja gráficos claros de la lluvia acumulada y avisa visualmente si el caudal sube demasiado para prevenir inundaciones.

2.3. Características de los usuarios

Tipo de usuario	Formación	Habilidades	Actividades
Analista Ambiental / Hidrólogo	Superior (Ingeniería Ambiental, Hidrología, Recursos Hídricos o afines).	Interpretación de regímenes de lluvia, cuencas hidrográficas, balances de caudal y análisis de series temporales.	Delimitar polígonos, ejecutar consultas por rango, comparar variabilidad pluvial y analizar curvas de precipitación.
Operador de Gestión de Riesgos / Protección Civil	Técnica o superior.	Lectura de paneles de control, interpretación de umbrales críticos y toma de decisiones preventivas.	Supervisión del mapa, monitoreo de estaciones y activación de protocolos ante eventos extremos.
Público General / Ciudadanía	No requiere formación específica.	Navegación web básica.	Consulta del estado meteorológico, zoom y desplazamiento en el mapa para conocer condiciones normales o alertas.

Tabla 3: Características de los usuarios.

2.4. Restricciones

1. El backend debe desarrollarse en Node.js (V8) con TypeScript, implementando el Quad-Tree de forma manual.
2. La recursión en el Call Stack debe limitarse para evitar desbordamientos, y el uso del Memory Heap debe optimizarse para no sobrecargar el Garbage Collector con objetos innecesarios.
3. La interfaz web debe construirse exclusivamente en Next.js, utilizando Leaflet para el mapa y D3.js para los gráficos dinámicos.
4. Requiere conexión a Internet para cargar las capas del mapa base y un navegador web moderno con soporte para JavaScript (ES6+) y renderizado SVG/Canvas.

2.5. Suposiciones y dependencias

Suposiciones:

1. Las estaciones del río Zamora tienen coordenadas fijas; solo cambian sus lecturas de lluvia y caudal en el tiempo.
2. Hay más estaciones en zonas urbanas que en montañas, justificando dividir el mapa según la densidad con el QuadTree.
3. Los valores de telemetría provienen de registros del INAMHI o de un simulador en formato JSON, sin requerir sensores físicos.
4. Se asume que las coordenadas y las lecturas climáticas ingresan con valores numéricos correctos.

Dependencias:

1. Entorno Node.js: requiere el motor V8 instalado para ejecutar el servidor y gestionar la memoria.
2. Servicio de mapas: depende de proveedores externos, como OpenStreetMap, para cargar el mapa base en Leaflet.
3. Librerías web: requiere los paquetes `leaflet` y `d3` funcionando dentro de Next.js.
4. Conexión de red: el usuario necesita acceso a la red para descargar el mapa y consultar los datos en el navegador.

2.6. Datos estáticos y dinámicos

Los datos utilizados por el sistema se clasifican en estáticos y dinámicos según su frecuencia de modificación, ciclo de vida y ubicación en memoria.

Tabla 4: Datos estáticos y dinámicos del sistema

Tipo	Datos	Almacenamiento (Stack / Heap)
Estáticos	• Coordenadas fijas (X,Y en WGS 84 UTM Zona 17S), altitud y códigos de las 13 estaciones del INAMHI: M0032 Santa Isabel, M0033 La Argelia, M0040 Pasaje, M0052 El Panguí, M0065 Macará Aeropuerto, M0072 Machala Aeropuerto, M0142 Saraguro, M0146 Cariamanga, M0148 Celica, M0180 Zaruma, M0185 Machala-UTM, M0138 Paute y M0139 Gualaceo [3]. • Límites perimetrales de la cuenca del río Zamora hasta la Central Hidroeléctrica Delsitanisagua (6.180,11 km²) [3]. • Parámetros invariables del QuadTree: capacidad máxima de elementos por cuadrante (K) y nivel de profundidad límite.	Las coordenadas numéricas primitivas (<code>number</code>), los valores constantes y las variables booleanas de control se evalúan directamente en la Pila (Call Stack) durante las funciones de inserción y comprobación espacial. La colección de referencia de las estaciones se mantiene como un arreglo inmutable en el Memory Heap.
Dinámicos	• Registros de telemetría y series de precipitación en tiempo real (valores entre 58,59 mm y 169,46 mm con sus marcas de tiempo) [3]. • Nodos y subdivisiones cuadráticas generadas dinámicamente (NW, NE, SW, SE) en el QuadTree. • Puntos espaciales recuperados tras ejecutar consultas por rango (Range Query) y coordenadas de los polígonos trazados interactivamente por el usuario.	Se almacenan en el Montón (Memory Heap) debido a su variabilidad de tamaño, creación continua en tiempo de ejecución y ciclo de vida dependiente de las consultas del usuario. Requieren una gestión eficiente de referencias para evitar pausas causadas por el recolector de basura (Garbage Collector) de Node.js.

3. Requerimientos específicos

3.1. Requerimientos funcionales

ID	Nombre	Descripción	Prioridad
RF-01	Inserción de estaciones	El sistema deberá permitir la inserción de las coordenadas geográficas de las estaciones meteorológicas dentro de la estructura Quad-Tree.	Alta
RF-02	Subdivisión de cuadrantes	El sistema deberá subdividir automáticamente un cuadrante del QuadTree en cuatro subcuadrantes (NW, NE, SW, SE) cuando se supere la capacidad máxima de puntos permitida.	Alta
RF-03	Actualización de telemetría	El sistema deberá recopilar y actualizar automáticamente las lecturas de lluvia utilizando datos simulados o precargados, descartando la adquisición directa de sensores en tiempo real.	Alta
RF-04	Selección de área en el mapa	El sistema deberá permitir al usuario dibujar o marcar un polígono o área rectangular directamente sobre el mapa interactivo.	Media
RF-05	Búsqueda por rango (Range Query)	El sistema deberá ejecutar una búsqueda espacial para recuperar las estaciones y datos contenidos dentro del área delimitada, descartando los puntos exteriores.	Alta
RF-06	Exploración de mapa interactivo	El sistema deberá presentar un mapa web mediante Leaflet que muestre la ubicación de las estaciones y permita zoom y desplazamiento.	Alta
RF-07	Generación de gráficos D3	El sistema deberá utilizar D3.js para dibujar gráficos de la lluvia acumulada y emitir alertas visuales si el caudal sube demasiado.	Media

Tabla 5: Requerimientos funcionales.

3.2. Requerimientos no funcionales

ID	Categoría	Descripción	Verificación
RNF-01	Desempeño de memoria	El uso de Heap del QuadTree deberá crecer de forma proporcional a la cantidad de puntos indexados.	Registrar <code>process.memoryUsage().heapUsed</code> antes y después con 10 000, 50 000 y 100 000 puntos.
RNF-02	Desempeño de memoria	Durante actualizaciones consecutivas, el uso de Heap no deberá crecer de forma sostenida una vez ejecutado el Garbage Collector.	Ejecutar 5 rondas de 10 000 actualizaciones con <code>node -expose-gc</code> y comparar <code>heapUsed</code> .
RNF-03	Rendimiento	En consultas sobre regiones pequeñas, Range Query deberá tardar menos que la misma consulta sobre un arreglo lineal con hasta 100 000 puntos.	Realizar 100 consultas sobre regiones de cerca del 5% del área y medir con <code>performance.now()</code> .
RNF-04	Escalabilidad	El tiempo promedio de inserción por punto deberá crecer de forma sublineal al aumentar la cantidad de puntos.	Construir con 10 000, 50 000 y 100 000 puntos y registrar tiempos.
RNF-05	Usabilidad	El dashboard deberá mostrar el resultado de una consulta por polígono en menos de 2 segundos después de que el usuario termine de dibujarlo.	Medir 10 consultas desde el navegador incluyendo red, backend y renderizado.
RNF-06	Mantenibilidad	El QuadTree deberá implementarse manualmente en TypeScript con modo <code>strict</code> y sin librerías externas para indexación espacial.	Compilar con <code>tsc -strict</code> y revisar <code>package.json</code> .

Tabla 6: Requerimientos no funcionales.

3.2.1. Análisis de complejidad

Sea n el número total de puntos, k el número de puntos que cumplen la consulta y d la profundidad que recorre el QuadTree.

Operación	QuadTree	Arreglo lineal
Inserción	$O(d)$	$O(1)$ al agregar al final
Búsqueda por rango	Depende de la región; una región pequeña visita pocos nodos además de los k resultados.	$O(n)$
Espacio en memoria	$O(n)$, contando puntos y nodos generados.	$O(n)$

Tabla 7: Complejidad del QuadTree frente a un arreglo lineal.

Si los puntos se reparten de forma razonablemente uniforme, d crece aproximadamente con el logaritmo de n . Con puntos muy agrupados, d aumenta y la inserción se vuelve más lenta, por eso la complejidad se expresa en función de d .

El arreglo inserta más rápido, pero cada consulta espacial obliga a revisar todos los puntos. El QuadTree gasta más trabajo al insertar porque debe subdividir el espacio, y lo recupera al consultar, ya que descarta regiones completas que no cruzan el área pedida. Como en este sistema las consultas por polígono son más frecuentes que las inserciones, esa diferencia justifica la estructura. La ventaja se reduce cuando la región consultada es grande, razón por la que RNF-03 se mide con regiones pequeñas.

Cada nodo del QuadTree es un objeto en el Heap que guarda los límites de su región y referencias a sus cuatro hijos (NW, NE, SW, SE). Por eso la memoria depende del número de nodos, además del número de puntos.

3.2.2. Referencia científica

Li et al. [5] estudian cómo indexar datos espaciales distribuidos de forma desigual, un caso parecido al de las estaciones del río Zamora, que se concentran más en zonas urbanas que en la montaña. Su índice divide el plano en celdas y mide la densidad de cada una. En un prototipo en memoria, la inserción fue más lenta que en los métodos de comparación, pero las consultas por rango fueron más rápidas. Este proyecto adopta el mismo principio con un QuadTree: un nodo se divide cuando supera su capacidad máxima de puntos, por lo que las zonas densas generan más subdivisiones. La relación entre el costo de inserción y la velocidad de consulta se medirá en RNF-03 y RNF-04.

3.2.3. Parámetros de diseño del QuadTree

El QuadTree tendrá una capacidad máxima de puntos por nodo y una profundidad máxima, ambas configurables. La capacidad define cuándo se subdivide un nodo: con una capacidad de 4, al llegar un quinto punto el nodo se divide en cuatro cuadrantes y reparte sus puntos. La profundidad máxima evita subdivisiones indefinidas cuando varios puntos tienen coordenadas iguales o muy cercanas, y limita la recursión en el Call Stack.

Ambos parámetros se podrán variar en las pruebas para observar su efecto en el tiempo de consulta y en el consumo de memoria.

3.3. Estructuras de datos a utilizar

En esta sección se completa la relación entre cada estructura y el manejo de memoria descrito en el documento.

Estructura	Uso en el sistema	Ubicación	Justificación
QuadTree	Índice espacial 2D para organizar puntos y realizar Range Query.	Heap	Sus nodos son objetos dinámicos con límites y referencias a cuatro hijos.
Arreglo	Almacenamiento de lecturas de sensores y colecciones de datos.	Heap	Su tamaño y contenido pueden variar durante la ejecución y requiere memoria dinámica.
Objeto / Clase	Representación de nodo, punto y región.	Heap	Las instancias son objetos complejos administrados por el Garbage Collector.
Tipos primitivos	Coordenadas, valores numéricos y banderas de control.	Stack	Los valores primitivos se manejan directamente en el contexto de ejecución de las funciones.

Tabla 8: Estructuras de datos y ubicación en memoria.

3.4. Interfaces externas

La interfaz de usuario se plantea mediante Next.js, Leaflet y D3.js. Leaflet se utilizará para la representación y exploración del mapa, mientras que D3.js permitirá generar los gráficos dinámicos y alertas visuales. La fuente de datos se basa en registros simulados o precargados, sin adquisición directa desde hardware de sensores.

Referencias

[1] K. Ravi, "Under the Hood: How V8 Manages the Stack and the Heap," *DEV Community*, Feb. 2024. [En línea]. Disponible en: https://dev.to/kiran_ravi_092a2cfcf60389/under-the-hood-how-v8-manages-the-stack-and-the-heap-nho

[2] MDN Web Docs, "Memory management," *Mozilla*, 2024. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Memory_management

- [3] A. R. Colcha Ortiz, N. E. Patiño Vaca, H. V. Pazmiño Chiliza, y J. P. Brito Noboa, "Análisis hidrológico y pluviométrico de la cuenca del río Zamora mediante SIG y Thiessen," *InvestiGO Revista Científica Multidisciplinaria*, vol. 7, n.º 20, pp. 890–907, jul.–dic. 2026. Disponible en: <https://www.revistainvestigo.com/EditorInvestigo/index.php/hm/article/view/486>. doi: <https://doi.org/10.56519/evp56r08>.
- [4] Y. Li, J. Yan, X. Huang, X. He, Z. Deng y Y. Chen, "R-MLGTI: A Grid- and R-Tree-Based Hybrid Index for Unevenly Distributed Spatial Data," *ISPRS International Journal of Geo-Information*, vol. 14, n.º 6, art. 231, jun. 2025. doi: <https://doi.org/10.3390/ijgi14060231>. Disponible en: <https://www.mdpi.com/2220-9964/14/6/231>.